

File Tree

해시 확인: certutil -hashfile <dist> SHA256

dist/

├─ assets/

│ └─ index-CNeejhjd.js (39025 B / f5db877b0ed8)

│ └─ index-DWoTo8jX.css (5855 B / 87cf11d43b33)

└─ index.html (3255 B / 553f0ec6681f)

==== assets/index-CNeejhjd.js (39025 B / f5db877b0ed8) ====

```
1 | const JLSTZ_90_KICKS = {
2 |   '0>1': [
3 |     { x: 0, y: 0 },
4 |     { x: -1, y: 0 },
5 |     { x: -1, y: 1 },
6 |     { x: 0, y: -2 },
7 |     { x: -1, y: -2 },
8 |   ],
9 |   '1>0': [
10 |    { x: 0, y: 0 },
11 |    { x: 1, y: 0 },
12 |    { x: 1, y: -1 },
13 |    { x: 0, y: 2 },
14 |    { x: 1, y: 2 },
15 |  ],
16 |   '1>2': [
17 |    { x: 0, y: 0 },
18 |    { x: 1, y: 0 },
19 |    { x: 1, y: -1 },
20 |    { x: 0, y: 2 },
21 |    { x: 1, y: 2 },
22 |  ],
23 |   '2>1': [
24 |    { x: 0, y: 0 },
25 |    { x: -1, y: 0 },
26 |    { x: -1, y: 1 },
27 |    { x: 0, y: -2 },
28 |    { x: -1, y: -2 },
29 |  ],
30 |   '2>3': [
31 |    { x: 0, y: 0 },
32 |    { x: 1, y: 0 },
33 |    { x: 1, y: 1 },
34 |    { x: 0, y: -2 },
35 |    { x: 1, y: -2 },
36 |  ],
37 |   '3>2': [
38 |    { x: 0, y: 0 },
39 |    { x: -1, y: 0 },
40 |    { x: -1, y: -1 },
41 |    { x: 0, y: 2 },
42 |    { x: -1, y: 2 },
43 |  ],
44 |   '3>0': [
45 |    { x: 0, y: 0 },
46 |    { x: -1, y: 0 },
47 |    { x: -1, y: -1 },
48 |    { x: 0, y: 2 },
49 |    { x: -1, y: 2 },
50 |  ],
51 |   '0>3': [
52 |    { x: 0, y: 0 },
53 |    { x: 1, y: 0 },
54 |    { x: 1, y: 1 },
55 |    { x: 0, y: -2 },
56 |    { x: 1, y: -2 },
57 |  ],
58 | },
59 | I_90_KICKS = {
60 |   '0>1': [
```

```

61 | | | | | {x: 0, y: 0},
62 | | | | | {x: -2, y: 0},
63 | | | | | {x: 1, y: 0},
64 | | | | | {x: -2, y: -1},
65 | | | | | {x: 1, y: 2},
66 | | | | | ],
67 | | | | | '1>0': [
68 | | | | | {x: 0, y: 0},
69 | | | | | {x: 2, y: 0},
70 | | | | | {x: -1, y: 0},
71 | | | | | {x: 2, y: 1},
72 | | | | | {x: -1, y: -2},
73 | | | | | ],
74 | | | | | '1>2': [
75 | | | | | {x: 0, y: 0},
76 | | | | | {x: -1, y: 0},
77 | | | | | {x: 2, y: 0},
78 | | | | | {x: -1, y: 2},
79 | | | | | {x: 2, y: -1},
80 | | | | | ],
81 | | | | | '2>1': [
82 | | | | | {x: 0, y: 0},
83 | | | | | {x: 1, y: 0},
84 | | | | | {x: -2, y: 0},
85 | | | | | {x: 1, y: -2},
86 | | | | | {x: -2, y: 1},
87 | | | | | ],
88 | | | | | '2>3': [
89 | | | | | {x: 0, y: 0},
90 | | | | | {x: 2, y: 0},
91 | | | | | {x: -1, y: 0},
92 | | | | | {x: 2, y: 1},
93 | | | | | {x: -1, y: -2},
94 | | | | | ],
95 | | | | | '3>2': [
96 | | | | | {x: 0, y: 0},
97 | | | | | {x: -2, y: 0},
98 | | | | | {x: 1, y: 0},
99 | | | | | {x: -2, y: -1},
100 | | | | | {x: 1, y: 2},
101 | | | | | ],
102 | | | | | '3>0': [
103 | | | | | {x: 0, y: 0},
104 | | | | | {x: 1, y: 0},
105 | | | | | {x: -2, y: 0},
106 | | | | | {x: 1, y: -2},
107 | | | | | {x: -2, y: 1},
108 | | | | | ],
109 | | | | | '0>3': [
110 | | | | | {x: 0, y: 0},
111 | | | | | {x: -1, y: 0},
112 | | | | | {x: 2, y: 0},
113 | | | | | {x: -1, y: 2},
114 | | | | | {x: 2, y: -1},
115 | | | | | ],
116 | | | | | },
117 | | | | | JLSTZ_180_KICKS = {
118 | | | | | '0>2': [
119 | | | | | {x: 0, y: 0},
120 | | | | | {x: 1, y: 0},
121 | | | | | {x: -1, y: 0},
122 | | | | | {x: 0, y: 1},
123 | | | | | {x: 1, y: 1},
124 | | | | | {x: -1, y: 1},
125 | | | | | {x: 0, y: -1},
126 | | | | | {x: 2, y: 0},
127 | | | | | {x: -2, y: 0},
128 | | | | | ],
129 | | | | | '2>0': [
130 | | | | | {x: 0, y: 0},
131 | | | | | {x: 1, y: 0},

```

```

132 |     {x: -1, y: 0},
133 |     {x: 0, y: -1},
134 |     {x: 1, y: -1},
135 |     {x: -1, y: -1},
136 |     {x: 0, y: 1},
137 |     {x: 2, y: 0},
138 |     {x: -2, y: 0},
139 | ],
140 | '1>3': [
141 |     {x: 0, y: 0},
142 |     {x: 0, y: 1},
143 |     {x: 0, y: -1},
144 |     {x: 1, y: 0},
145 |     {x: -1, y: 0},
146 |     {x: 2, y: 0},
147 |     {x: -2, y: 0},
148 |     {x: 1, y: -1},
149 |     {x: -1, y: -1},
150 | ],
151 | '3>1': [
152 |     {x: 0, y: 0},
153 |     {x: 0, y: 1},
154 |     {x: 0, y: -1},
155 |     {x: 1, y: 0},
156 |     {x: -1, y: 0},
157 |     {x: 2, y: 0},
158 |     {x: -2, y: 0},
159 |     {x: 1, y: 1},
160 |     {x: -1, y: 1},
161 | ],
162 | },
163 | I_180_KICKS = {
164 |     '0>2': [
165 |         {x: 0, y: 0},
166 |         {x: -1, y: 0},
167 |         {x: 2, y: 0},
168 |         {x: -1, y: 2},
169 |         {x: 2, y: -1},
170 |         {x: 0, y: 1},
171 |         {x: 0, y: -1},
172 |     ],
173 |     '2>0': [
174 |         {x: 0, y: 0},
175 |         {x: 1, y: 0},
176 |         {x: -2, y: 0},
177 |         {x: 1, y: -2},
178 |         {x: -2, y: 1},
179 |         {x: 0, y: 1},
180 |         {x: 0, y: -1},
181 |     ],
182 |     '1>3': [
183 |         {x: 0, y: 0},
184 |         {x: 0, y: 1},
185 |         {x: 0, y: -1},
186 |         {x: 1, y: 0},
187 |         {x: -2, y: 0},
188 |         {x: 1, y: 2},
189 |         {x: -2, y: -1},
190 |     ],
191 |     '3>1': [
192 |         {x: 0, y: 0},
193 |         {x: 0, y: 1},
194 |         {x: 0, y: -1},
195 |         {x: -1, y: 0},
196 |         {x: 2, y: 0},
197 |         {x: -1, y: -2},
198 |         {x: 2, y: 1},
199 |     ],
200 | },
201 | O_KICK = [{x: 0, y: 0}],
202 | CW_ROTATION = [1, 2, 3, 0],

```

```

203 | ·CCW_ROTATION = [3, 0, 1, 2],
204 | ·DOUBLE_ROTATION = [2, 3, 0, 1],
205 | ·getTargetRotation = (rotation, direction) =>
206 |   ··'cw' === direction
207 |   ··? CW_ROTATION[rotation]
208 |   ··: 'ccw' === direction
209 |   ··? CCW_ROTATION[rotation]
210 |   ··: DOUBLE_ROTATION[rotation],
211 | ·LINE_CLEAR_BASE = { 0: 0, 1: 100, 2: 300, 3: 500, 4: 800 },
212 | ·TSPIN_BASE = { 0: 400, 1: 800, 2: 1200, 3: 1600 },
213 | ·TSPIN_MINI_BASE = { 0: 100, 1: 200, 2: 400 },
214 | ·PERFECT_CLEAR_BASE = { 1: 800, 2: 1200, 3: 1800, 4: 2e3 },
215 | ·isDifficultClear = (lines, tspin) => !(lines <= 0 || (4 !== lines && 'none' === tspin)),
216 | ·DAS_RANGE = { min: 0, max: 300 },
217 | ·ARR_RANGE = { min: 0, max: 100 },
218 | ·SDF_RANGE = { min: 1, max: 40 },
219 | ·DEFAULT_HANDLING = { dasMs: 100, arrMs: 0, sdfG: 20 },
220 | ·clamp = (value, min, max) => Math.min(Math.max(value, min), max),
221 | ·normalizeHandling = (partial) => ({
222 |   ·dasMs: clamp(
223 |     ·Number.isFinite(partial.dasMs)
224 |     ·? Math.round(partial.dasMs ?? DEFAULT_HANDLING.dasMs)
225 |     ·: DEFAULT_HANDLING.dasMs,
226 |   ·DAS_RANGE.min,
227 |   ·DAS_RANGE.max,
228 |   ·),
229 |   ·arrMs: clamp(
230 |     ·Number.isFinite(partial.arrMs)
231 |     ·? Math.round(partial.arrMs ?? DEFAULT_HANDLING.arrMs)
232 |     ·: DEFAULT_HANDLING.arrMs,
233 |   ·ARR_RANGE.min,
234 |   ·ARR_RANGE.max,
235 |   ·),
236 |   ·sdfG: clamp(
237 |     ·Number.isFinite(partial.sdfG)
238 |     ·? Number(partial.sdfG ?? DEFAULT_HANDLING.sdfG)
239 |     ·: DEFAULT_HANDLING.sdfG,
240 |   ·SDF_RANGE.min,
241 |   ·SDF_RANGE.max,
242 |   ·),
243 |   ·}),
244 | ·handlingRanges_dasMs = DAS_RANGE,
245 | ·handlingRanges_arrMs = ARR_RANGE,
246 | ·handlingRanges_sdfG = SDF_RANGE,
247 | ·PIECE_ORDER = ['I', 'O', 'T', 'S', 'Z', 'J', 'L'],
248 | ·createEmptyRow = () => Array.from({ length: 10 }, () => null),
249 | ·createShuffledBag = (random) => {
250 |   ·const bag = [...PIECE_ORDER];
251 |   ·for (let index = bag.length - 1; index > 0; index -= 1) {
252 |     ·const swapIndex = Math.floor(random() * (index + 1)),
253 |     ·current = bag[index];
254 |     ·((bag[index] = bag[swapIndex]), (bag[swapIndex] = current));
255 |   ·}
256 |   ·return bag;
257 | },
258 | ·PIECE_CELLS = {
259 |   ·I: [
260 |     ·[
261 |       ·{ x: 0, y: 1 },
262 |       ·{ x: 1, y: 1 },
263 |       ·{ x: 2, y: 1 },
264 |       ·{ x: 3, y: 1 },
265 |     ],
266 |     ·[
267 |       ·{ x: 2, y: 0 },
268 |       ·{ x: 2, y: 1 },
269 |       ·{ x: 2, y: 2 },
270 |       ·{ x: 2, y: 3 },
271 |     ],
272 |     ·[
273 |       ·{ x: 0, y: 2 },

```

```

274 | . | . | . | { x: 1, y: 2 },
275 | . | . | . | { x: 2, y: 2 },
276 | . | . | . | { x: 3, y: 2 },
277 | . | . | . | ],
278 | . | . | . | [
279 | . | . | . | { x: 1, y: 0 },
280 | . | . | . | { x: 1, y: 1 },
281 | . | . | . | { x: 1, y: 2 },
282 | . | . | . | { x: 1, y: 3 },
283 | . | . | . | ],
284 | . | . | . | ],
285 | . | . | 0: [
286 | . | . | . | [
287 | . | . | . | { x: 1, y: 0 },
288 | . | . | . | { x: 2, y: 0 },
289 | . | . | . | { x: 1, y: 1 },
290 | . | . | . | { x: 2, y: 1 },
291 | . | . | . | ],
292 | . | . | . | [
293 | . | . | . | { x: 1, y: 0 },
294 | . | . | . | { x: 2, y: 0 },
295 | . | . | . | { x: 1, y: 1 },
296 | . | . | . | { x: 2, y: 1 },
297 | . | . | . | ],
298 | . | . | . | [
299 | . | . | . | { x: 1, y: 0 },
300 | . | . | . | { x: 2, y: 0 },
301 | . | . | . | { x: 1, y: 1 },
302 | . | . | . | { x: 2, y: 1 },
303 | . | . | . | ],
304 | . | . | . | [
305 | . | . | . | { x: 1, y: 0 },
306 | . | . | . | { x: 2, y: 0 },
307 | . | . | . | { x: 1, y: 1 },
308 | . | . | . | { x: 2, y: 1 },
309 | . | . | . | ],
310 | . | . | . | ],
311 | . | . | T: [
312 | . | . | . | [
313 | . | . | . | { x: 1, y: 0 },
314 | . | . | . | { x: 0, y: 1 },
315 | . | . | . | { x: 1, y: 1 },
316 | . | . | . | { x: 2, y: 1 },
317 | . | . | . | ],
318 | . | . | . | [
319 | . | . | . | { x: 1, y: 0 },
320 | . | . | . | { x: 1, y: 1 },
321 | . | . | . | { x: 2, y: 1 },
322 | . | . | . | { x: 1, y: 2 },
323 | . | . | . | ],
324 | . | . | . | [
325 | . | . | . | { x: 0, y: 1 },
326 | . | . | . | { x: 1, y: 1 },
327 | . | . | . | { x: 2, y: 1 },
328 | . | . | . | { x: 1, y: 2 },
329 | . | . | . | ],
330 | . | . | . | [
331 | . | . | . | { x: 1, y: 0 },
332 | . | . | . | { x: 0, y: 1 },
333 | . | . | . | { x: 1, y: 1 },
334 | . | . | . | { x: 1, y: 2 },
335 | . | . | . | ],
336 | . | . | . | ],
337 | . | . | S: [
338 | . | . | . | [
339 | . | . | . | { x: 1, y: 0 },
340 | . | . | . | { x: 2, y: 0 },
341 | . | . | . | { x: 0, y: 1 },
342 | . | . | . | { x: 1, y: 1 },
343 | . | . | . | ],
344 | . | . | . | [

```

```

345 | . | . | . | { x: 1, y: 0 },
346 | . | . | . | { x: 1, y: 1 },
347 | . | . | . | { x: 2, y: 1 },
348 | . | . | . | { x: 2, y: 2 },
349 | . | . | . | ],
350 | . | . | . | [
351 | . | . | . | { x: 1, y: 1 },
352 | . | . | . | { x: 2, y: 1 },
353 | . | . | . | { x: 0, y: 2 },
354 | . | . | . | { x: 1, y: 2 },
355 | . | . | . | ],
356 | . | . | . | [
357 | . | . | . | { x: 0, y: 0 },
358 | . | . | . | { x: 0, y: 1 },
359 | . | . | . | { x: 1, y: 1 },
360 | . | . | . | { x: 1, y: 2 },
361 | . | . | . | ],
362 | . | . | . | ],
363 | . | . | . | Z: [
364 | . | . | . | [
365 | . | . | . | { x: 0, y: 0 },
366 | . | . | . | { x: 1, y: 0 },
367 | . | . | . | { x: 1, y: 1 },
368 | . | . | . | { x: 2, y: 1 },
369 | . | . | . | ],
370 | . | . | . | [
371 | . | . | . | { x: 2, y: 0 },
372 | . | . | . | { x: 1, y: 1 },
373 | . | . | . | { x: 2, y: 1 },
374 | . | . | . | { x: 1, y: 2 },
375 | . | . | . | ],
376 | . | . | . | [
377 | . | . | . | { x: 0, y: 1 },
378 | . | . | . | { x: 1, y: 1 },
379 | . | . | . | { x: 1, y: 2 },
380 | . | . | . | { x: 2, y: 2 },
381 | . | . | . | ],
382 | . | . | . | [
383 | . | . | . | { x: 1, y: 0 },
384 | . | . | . | { x: 0, y: 1 },
385 | . | . | . | { x: 1, y: 1 },
386 | . | . | . | { x: 0, y: 2 },
387 | . | . | . | ],
388 | . | . | . | ],
389 | . | . | . | J: [
390 | . | . | . | [
391 | . | . | . | { x: 0, y: 0 },
392 | . | . | . | { x: 0, y: 1 },
393 | . | . | . | { x: 1, y: 1 },
394 | . | . | . | { x: 2, y: 1 },
395 | . | . | . | ],
396 | . | . | . | [
397 | . | . | . | { x: 1, y: 0 },
398 | . | . | . | { x: 2, y: 0 },
399 | . | . | . | { x: 1, y: 1 },
400 | . | . | . | { x: 1, y: 2 },
401 | . | . | . | ],
402 | . | . | . | [
403 | . | . | . | { x: 0, y: 1 },
404 | . | . | . | { x: 1, y: 1 },
405 | . | . | . | { x: 2, y: 1 },
406 | . | . | . | { x: 2, y: 2 },
407 | . | . | . | ],
408 | . | . | . | [
409 | . | . | . | { x: 1, y: 0 },
410 | . | . | . | { x: 1, y: 1 },
411 | . | . | . | { x: 0, y: 2 },
412 | . | . | . | { x: 1, y: 2 },
413 | . | . | . | ],
414 | . | . | . | ],
415 | . | . | . | L: [

```

```

416 | | | | | [
417 | | | | | { x: 2, y: 0 },
418 | | | | | { x: 0, y: 1 },
419 | | | | | { x: 1, y: 1 },
420 | | | | | { x: 2, y: 1 },
421 | | | | | ],
422 | | | | | [
423 | | | | | { x: 1, y: 0 },
424 | | | | | { x: 1, y: 1 },
425 | | | | | { x: 1, y: 2 },
426 | | | | | { x: 2, y: 2 },
427 | | | | | ],
428 | | | | | [
429 | | | | | { x: 0, y: 1 },
430 | | | | | { x: 1, y: 1 },
431 | | | | | { x: 2, y: 1 },
432 | | | | | { x: 0, y: 2 },
433 | | | | | ],
434 | | | | | [
435 | | | | | { x: 0, y: 0 },
436 | | | | | { x: 1, y: 0 },
437 | | | | | { x: 1, y: 1 },
438 | | | | | { x: 1, y: 2 },
439 | | | | | ],
440 | | | | | ],
441 | | | },
442 | | pieceCellsFor = (pieceType, rotation) => PIECE_CELLS[pieceType][rotation],
443 | | getAbsoluteCells = (piece) =>
444 | |   pieceCellsFor(piece.type, piece.rotation).map((cell) => ({
445 | |     x: piece.x + cell.x,
446 | |     y: piece.y + cell.y,
447 | |   })),
448 | | createActivePiece = (pieceType) => ({
449 | |   type: pieceType,
450 | |   x: 3,
451 | |   y: 18,
452 | |   rotation: 0,
453 | |   lockDelayMs: 0,
454 | |   lockResets: 0,
455 | |   lastAction: 'none',
456 | |   lastKickIndex: null,
457 | |   softDropCells: 0,
458 | |   hardDropCells: 0,
459 | | }),
460 | | isCellBlocked = (board, x, y) =>
461 | |   !!((x, y) => x < 0 || x >= 10 || y >= 40)(x, y) || (!(y < 0) && null !== board[y][x]),
462 | | collides = (board, piece) =>
463 | |   getAbsoluteCells(piece).some((cell) => isCellBlocked(board, cell.x, cell.y)),
464 | | isGrounded = (board, piece) => {
465 | |   const moved = { ... piece, y: piece.y + 1 };
466 | |   return collides(board, moved);
467 | | },
468 | | normalizeVisiblePoint = (point) => {
469 | |   const visibleY = point.y - 20;
470 | |   return visibleY < 0 || visibleY >= 20 ? null : { x: point.x, y: visibleY };
471 | | },
472 | | computeGhostCells = (board, active) => {
473 | |   if (!active) return [];
474 | |   const ghost = { ... active };
475 | |   for (; !collides(board, { ... ghost, y: ghost.y + 1 }); ) ghost.y += 1;
476 | |   return getAbsoluteCells(ghost)
477 | |     .map(normalizeVisiblePoint)
478 | |     .filter((point) => null !== point);
479 | | },
480 | | createInitialState = (random, handling = DEFAULT_HANDLING) => ({
481 | |   status: 'ready',
482 | |   board: Array.from({ length: 40 }, () => createEmptyRow()),
483 | |   active: null,
484 | |   hold: null,
485 | |   canHold: !0,
486 | |   queue: [],

```

```

487 |     bag: [],
488 |     score: 0,
489 |     lines: 0,
490 |     level: 1,
491 |     goal: 10,
492 |     combo: -1,
493 |     b2bChain: 0,
494 |     handling: handling,
495 |     softDropActive: !1,
496 |     gravityCarry: 0,
497 |     random: random,
498 |   })),
499 |   HANDLED_CODES = new Set([
500 |     'ArrowLeft',
501 |     'ArrowRight',
502 |     'ArrowDown',
503 |     'ArrowUp',
504 |     'Space',
505 |     'KeyZ',
506 |     'KeyX',
507 |     'KeyA',
508 |     'KeyC',
509 |     'ShiftLeft',
510 |     'ShiftRight',
511 |     'KeyP',
512 |     'KeyR',
513 |     'Enter',
514 |   ]),
515 |   PREVIEW_CELLS = {
516 |     I: [
517 |       {x: 0, y: 1},
518 |       {x: 1, y: 1},
519 |       {x: 2, y: 1},
520 |       {x: 3, y: 1},
521 |     ],
522 |     O: [
523 |       {x: 1, y: 1},
524 |       {x: 2, y: 1},
525 |       {x: 1, y: 2},
526 |       {x: 2, y: 2},
527 |     ],
528 |     T: [
529 |       {x: 1, y: 0},
530 |       {x: 0, y: 1},
531 |       {x: 1, y: 1},
532 |       {x: 2, y: 1},
533 |     ],
534 |     S: [
535 |       {x: 1, y: 0},
536 |       {x: 2, y: 0},
537 |       {x: 0, y: 1},
538 |       {x: 1, y: 1},
539 |     ],
540 |     Z: [
541 |       {x: 0, y: 0},
542 |       {x: 1, y: 0},
543 |       {x: 1, y: 1},
544 |       {x: 2, y: 1},
545 |     ],
546 |     J: [
547 |       {x: 0, y: 0},
548 |       {x: 0, y: 1},
549 |       {x: 1, y: 1},
550 |       {x: 2, y: 1},
551 |     ],
552 |     L: [
553 |       {x: 2, y: 0},
554 |       {x: 0, y: 1},
555 |       {x: 1, y: 1},
556 |       {x: 2, y: 1},
557 |     ],

```



```

558 | },
559 | toneClass = (pieceType) => `tone-${pieceType.toLowerCase()}`,
560 | createCellGrid = (container, count, className) => {
561 |   const fragment = document.createDocumentFragment(),
562 |   cells = [];
563 |   for (let index = 0; index < count; index += 1) {
564 |     const cell = document.createElement('div');
565 |     ((cell.className = className),
566 |     cell.setAttribute('aria-hidden', 'true'),
567 |     fragment.append(cell),
568 |     cells.push(cell));
569 |   }
570 |   return (container.replaceChildren(fragment), cells);
571 | },
572 | setText = (node, value) => {
573 |   node.textContent === value && (node.textContent = value);
574 | },
575 | initialHandling = (() => {
576 |   try {
577 |     const raw = globalThis.localStorage?.getItem('tetris_handling_v1');
578 |     if (!raw) return { ... DEFAULT_HANDLING };
579 |     const parsed = JSON.parse(raw);
580 |     return normalizeHandling(parsed);
581 |   } catch {
582 |     return { ... DEFAULT_HANDLING };
583 |   }
584 | })();
585 | let highScore = (() => {
586 |   try {
587 |     const raw = globalThis.localStorage?.getItem('tetris_high_score_v1'),
588 |     parsed = raw ? Number(raw) : 0;
589 |     return Number.isFinite(parsed) ? Math.max(0, Math.floor(parsed)) : 0;
590 |   } catch {
591 |     return 0;
592 |   }
593 | })();
594 | const engine = ((config) => {
595 |   let state = createInitialState(
596 |     config?.random ?? Math.random,
597 |     normalizeHandling(config?.handling ?? {}),
598 |   );
599 |   const fillQueue = (targetLength) => {
600 |     for (; state.queue.length < targetLength; ) {
601 |       0 === state.bag.length && (state.bag = createShuffledBag(state.random));
602 |       const nextType = state.bag.shift();
603 |       if (!nextType) break;
604 |       state.queue.push(nextType);
605 |     }
606 |   },
607 |   spawnFromQueue = () => {
608 |     fillQueue(6);
609 |     const nextType = state.queue.shift();
610 |     if (!nextType) return !1;
611 |     const spawned = createActivePiece(nextType);
612 |     return collides(state.board, spawned)
613 |       ? ((state.active = null), (state.status = 'gameover'), !1)
614 |       : ((state.active = spawned), !0);
615 |   },
616 |   resetForNewRun = () => {
617 |     ((state = createInitialState(state.random, state.handling)),
618 |     fillQueue(6),
619 |     spawnFromQueue(),
620 |     (state.status = 'running'));
621 |   },
622 |   maybeResetLockTimer = () => {
623 |     state.active &&
624 |     (state.active.lockResets >= 15 ||
625 |     ((state.active.lockDelayMs = 0), (state.active.lockResets += 1)));
626 |   },
627 |   wouldLeaveGroundAfterLockResets = (wasGrounded, next) =>
628 |   wasGrounded &&

```

```

629 |         null !== state.active &&
630 |         state.active.lockResets >= 15 &&
631 |         !isGrounded(state.board, next),
632 |         tryShift = (dx, dy, fromPlayerInput) => {
633 |             if (!state.active) return !1;
634 |             const next = { ... state.active, x: state.active.x + dx, y: state.active.y + dy };
635 |             if (collides(state.board, next)) return !1;
636 |             const wasGrounded = isGrounded(state.board, state.active);
637 |             return !(
638 |                 (fromPlayerInput && wouldLeaveGroundAfterLockResets(wasGrounded, next)) ||
639 |                 ((state.active.x = next.x),
640 |                 (state.active.y = next.y),
641 |                 (state.active.lastKickIndex = null),
642 |                 0 !== dy
643 |                 ? (state.active.lastAction = 'none')
644 |                 : fromPlayerInput && (state.active.lastAction = 'move'),
645 |                 fromPlayerInput && wasGrounded && maybeResetLockTimer(),
646 |                 0)
647 |             );
648 |         },
649 |         lockActivePiece = () => {
650 |             if (!state.active) return;
651 |             const lockedPiece = state.active;
652 |             for (const cell of getAbsoluteCells(lockedPiece))
653 |                 cell.y >= 0 &&
654 |                 cell.y < 40 &&
655 |                 cell.x >= 0 &&
656 |                 cell.x < 10 &&
657 |                 (state.board[cell.y][cell.x] = lockedPiece.type);
658 |             const tspin = ((board, piece) => {
659 |                 if ('T' !== piece.type || 'rotate' !== piece.lastAction) return 'none';
660 |                 const pivot_x = piece.x + 1,
661 |                     pivot_y = piece.y + 1,
662 |                     corners = [
663 |                         { x: pivot_x - 1, y: pivot_y - 1 },
664 |                         { x: pivot_x + 1, y: pivot_y - 1 },
665 |                         { x: pivot_x - 1, y: pivot_y + 1 },
666 |                         { x: pivot_x + 1, y: pivot_y + 1 },
667 |                     ];
668 |                 if (corners.filter((corner) => isCellBlocked(board, corner.x, corner.y)).length < 3)
669 |                     return 'none';
670 |                 const [firstFront, secondFront] = [{ 0: [0, 1], 1: [1, 3], 2: [2, 3], 3: [0, 2] },
671 |                 piece.rotation
672 |                 ];
673 |                 return 2 ===
674 |                 [corners[firstFront], corners[secondFront]].filter((corner) =>
675 |                     isCellBlocked(board, corner.x, corner.y),
676 |                     ).length
677 |                 ? 'full'
678 |                 : 'mini';
679 |             })(state.board, lockedPiece),
680 |             linesCleared = ((board) => {
681 |                 const filledRows = [];
682 |                 for (let y = 0; y < 40; y += 1)
683 |                     board[y].every((cell) => null !== cell) && filledRows.push(y);
684 |                 if (0 === filledRows.length) return 0;
685 |                 for (const rowIndex of [... filledRows].reverse())
686 |                     board.splice(rowIndex, 1), board.unshift(createEmptyRow());
687 |                 return filledRows.length;
688 |             })(state.board),
689 |             perfectClear = state.board.every((row) => row.every((cell) => null === cell));
690 |             linesCleared > 0 ? (state.combo += 1) : (state.combo = -1);
691 |             const scoreBreakdown = ((event) => {
692 |                 const level = Math.max(event.level, 1),
693 |                     base =
694 |                     ((lines = event.lines),
695 |                     ('full' === (tspin = event.tspin)
696 |                     ? (TSPIN_BASE[lines] ?? 0)
697 |                     : 'mini' === tspin
698 |                     ? (TSPIN_MINI_BASE[lines] ?? 0)
699 |                     : (LINE_CLEAR_BASE[lines] ?? 0)) * level);

```

```

700 |         var lines, tspin;
701 |         const difficult = isDifficultClear(event.lines, event.tspin),
702 |             keepsB2B = ((lines, tspin) =>
703 |                 !isDifficultClear(lines, tspin) || (0 === lines && 'none' !== tspin))(
704 |                 event.lines,
705 |                 event.tspin,
706 |             ),
707 |             b2bBonus = difficult && event.b2bActive ? Math.floor(0.5 * base) : 0,
708 |             comboBonus = event.lines > 0 && event.combo > 0 ? 50 * event.combo * level : 0;
709 |         let perfectClearBonus = 0;
710 |         if (event.perfectClear && event.lines > 0) {
711 |             perfectClearBonus =
712 |                 (4 === event.lines && event.b2bActive
713 |                 ? 3200
714 |                 : (PERFECT_CLEAR_BASE[event.lines] ?? 0)) * level;
715 |         }
716 |         const dropBonus = event.softDropCells + 2 * event.hardDropCells;
717 |         return {
718 |             base: base,
719 |             b2bBonus: b2bBonus,
720 |             comboBonus: comboBonus,
721 |             perfectClearBonus: perfectClearBonus,
722 |             dropBonus: dropBonus,
723 |             total: base + b2bBonus + comboBonus + perfectClearBonus + dropBonus,
724 |             difficult: difficult,
725 |             keepsB2B: keepsB2B,
726 |         };
727 |     }({
728 |         level: state.level,
729 |         lines: linesCleared,
730 |         tspin: tspin,
731 |         b2bActive: state.b2bChain > 0,
732 |         combo: state.combo,
733 |         perfectClear: perfectClear,
734 |         softDropCells: lockedPiece.softDropCells,
735 |         hardDropCells: lockedPiece.hardDropCells,
736 |     });
737 |     var lines, level;
738 |     ((state.score += scoreBreakdown.total),
739 |     linesCleared > 0 &&
740 |     (scoreBreakdown.difficult ? (state.b2bChain += 1) : (state.b2bChain = 0)),
741 |     (state.lines += linesCleared),
742 |     (state.level =
743 |         (lines = state.lines) < 150
744 |         ? Math.floor(lines / 10) + 1
745 |         : 16 + Math.floor((lines - 150) / 10)),
746 |     (state.goal = (level = state.level) ≤ 15 ? 10 * level : 150 + 10 * (level - 15)),
747 |     (state.canHold = !0),
748 |     (state.gravityCarry = 0),
749 |     spawnFromQueue());
750 | },
751 | tryRotate = (direction) => {
752 |     if (!state.active) return !1;
753 |     const wasGrounded = isGrounded(state.board, state.active),
754 |         result = (({
755 |             pieceType: pieceType,
756 |             x: x,
757 |             y: y,
758 |             rotation: rotation,
759 |             direction: direction,
760 |             cellsFor: cellsFor,
761 |             isBlocked: isBlocked,
762 |         }) => {
763 |             const targetRotation = getTargetRotation(rotation, direction),
764 |                 targetCells = cellsFor(pieceType, targetRotation),
765 |                 kickTests = ((pieceType, from, direction) => {
766 |                     const to = getTargetRotation(from, direction);
767 |                     if ('O' === pieceType) return 0_KICK;
768 |                     const key = ((from, to) => `${from}>${to}`)(from, to);
769 |                     return '180' === direction
770 |                         ? 'I' === pieceType

```

```

771 |         ? (I_180_KICKS[key] ?? 0_KICK)
772 |         : (JLSTZ_180_KICKS[key] ?? 0_KICK)
773 |         : 'I' === pieceType
774 |         ? (I_90_KICKS[key] ?? 0_KICK)
775 |         : (JLSTZ_90_KICKS[key] ?? 0_KICK);
776 |     })(pieceType, rotation, direction);
777 |     for (let index = 0; index < kickTests.length; index += 1) {
778 |         const kick = kickTests[index],
779 |               testX = x + kick.x,
780 |               testY = y + kick.y;
781 |         if (!targetCells.some((cell) => isBlocked(testX + cell.x, testY + cell.y)))
782 |             return {
783 |                 success: !0,
784 |                 x: testX,
785 |                 y: testY,
786 |                 rotation: targetRotation,
787 |                 kickIndex: index,
788 |             };
789 |     }
790 |     return { success: !1, x: x, y: y, rotation: rotation, kickIndex: null };
791 | } ({
792 |     pieceType: state.active.type,
793 |     x: state.active.x,
794 |     y: state.active.y,
795 |     rotation: state.active.rotation,
796 |     direction: direction,
797 |     cellsFor: pieceCellsFor,
798 |     isBlocked: (x, y) => isCellBlocked(state.board, x, y),
799 | });
800 | if (!result.success) return !1;
801 | const next = { ... state.active, x: result.x, y: result.y, rotation: result.rotation };
802 | return (
803 |     !wouldLeaveGroundAfterLockResets(wasGrounded, next) &&
804 |     ((state.active.x = result.x),
805 |     (state.active.y = result.y),
806 |     (state.active.rotation = result.rotation),
807 |     (state.active.lastAction = 'rotate'),
808 |     (state.active.lastKickIndex = result.kickIndex),
809 |     wasGrounded && maybeResetLockTimer(),
810 |     !0)
811 | );
812 | };
813 | return (
814 |     fillQueue(6),
815 |     {
816 |         dispatch: (action) =>
817 |             'setHandling' === action.type
818 |             ? ((state.handling = normalizeHandling(action.payload)), !0)
819 |             : 'restart' === action.type
820 |             ? (resetForNewRun(), !0)
821 |             : 'start' === action.type
822 |             ? 'paused' === state.status
823 |             ? ((state.status = 'running'), !0)
824 |             : ('ready' === state.status || 'gameover' === state.status) &&
825 |             (resetForNewRun(), !0)
826 |             : 'togglePause' === action.type
827 |             ? 'running' === state.status
828 |             ? ((state.status = 'paused'), !0)
829 |             : 'paused' === state.status && ((state.status = 'running'), !0)
830 |             : 'running' === state.status &&
831 |             ((action) => {
832 |                 switch (action.type) {
833 |                     case 'moveLeft':
834 |                         return tryShift(-1, 0, !0);
835 |                     case 'moveRight':
836 |                         return tryShift(1, 0, !0);
837 |                     case 'rotateCW':
838 |                         return tryRotate('cw');
839 |                     case 'rotateCCW':
840 |                         return tryRotate('ccw');
841 |                     case 'rotate180':

```

```

842 |         return tryRotate('180');
843 |     case 'hardDrop':
844 |         return (() => {
845 |             if (!state.active) return !1;
846 |             let dropped = 0;
847 |             for (; tryShift(0, 1, !1);) dropped += 1;
848 |             return (
849 |                 (state.active.hardDropCells += dropped),
850 |                 dropped > 0 && (state.active.lastAction = 'none'),
851 |                 lockActivePiece(),
852 |                 !0
853 |             );
854 |         })();
855 |     case 'hold':
856 |         return (() => {
857 |             if (!state.active || !state.canHold) return !1;
858 |             const currentType = state.active.type;
859 |             if (state.hold) {
860 |                 const swapType = state.hold;
861 |                 if (
862 |                     ((state.hold = currentType),
863 |                     (state.active = createActivePiece(swapType)),
864 |                     collides(state.board, state.active))
865 |                 )
866 |                     return ((state.active = null), (state.status = 'gameover'), !1);
867 |                 } else
868 |                     ((state.hold = currentType), (state.active = null), spawnFromQueue());
869 |             return ((state.canHold = !1), (state.gravityCarry = 0), !0);
870 |         })();
871 |     default:
872 |         return !1;
873 |     }
874 | })(action),
875 | tick: (dtMs) => {
876 |     if ('running' !== state.status || !state.active) return;
877 |     const gravity = ((level) => {
878 |         const base = [
879 |             0, 0.01667, 0.021017, 0.026977, 0.035256, 0.04693, 0.06361, 0.0879, 0.1236, 0.1775,
880 |             0.2598, 0.388, 0.59, 0.92, 1.46, 2.36,
881 |         ];
882 |         if (level <= 15) return base[Math.max(level, 1)];
883 |         let gravity = base[15];
884 |         for (let lv = 16; lv <= level; lv += 1) gravity *= 1.2;
885 |         return gravity;
886 |     })(state.level),
887 |     effectiveGravity = state.softDropActive
888 |     ? Math.max(gravity, state.handling.sdfG)
889 |     : gravity;
890 |     for (
891 |         state.gravityCarry += (effectiveGravity * dtMs) / 16.666666666666668;
892 |         state.gravityCarry >= 1 && state.active;
893 |     ) {
894 |         if (!tryShift(0, 1, !1)) {
895 |             state.gravityCarry = 0;
896 |             break;
897 |         }
898 |         (state.softDropActive && (state.active.softDropCells += 1), (state.gravityCarry -= 1));
899 |     }
900 |     state.active &&
901 |     (isGrounded(state.board, state.active)
902 |     ? ((state.active.lockDelayMs += dtMs),
903 |     state.active.lockDelayMs >= 500 && lockActivePiece())
904 |     : (state.active.lockDelayMs = 0));
905 | },
906 | setSoftDropActive: (active) => {
907 |     state.softDropActive = active;
908 | },
909 | getSnapshot: () => {
910 |     const visibleBoard = state.board.slice(20).map((row) => [... row]),
911 |     activeCells = state.active
912 |     ? getAbsoluteCells(state.active)

```

```

913 |         .map(normalizeVisiblePoint)
914 |         .filter((point) => null !== point)
915 |         : [];
916 |     return {
917 |         status: state.status,
918 |         boardVisible: visibleBoard,
919 |         activeCells: activeCells,
920 |         activeType: state.active?.type ?? null,
921 |         ghostCells: computeGhostCells(state.board, state.active),
922 |         hold: state.hold,
923 |         next: state.queue.slice(0, 5),
924 |         score: state.score,
925 |         level: state.level,
926 |         lines: state.lines,
927 |         goal: state.goal,
928 |         combo: state.combo,
929 |         b2bChain: state.b2bChain,
930 |         handling: { ... state.handling },
931 |     };
932 | },
933 | getHandling: () => ({ ... state.handling }),
934 | }
935 | );
936 | })( { handling: initialHandling },
937 | renderer = (() => {
938 |     const boardGrid = document.querySelector('#board-grid'),
939 |     nextGrids = [ ... document.querySelectorAll('.next-grid') ],
940 |     holdGrid = document.querySelector('#hold-grid'),
941 |     scoreValue = document.querySelector('#score-value'),
942 |     highScoreValue = document.querySelector('#high-score-value'),
943 |     levelValue = document.querySelector('#level-value'),
944 |     linesValue = document.querySelector('#lines-value'),
945 |     goalValue = document.querySelector('#goal-value'),
946 |     comboToast = document.querySelector('#combo-toast'),
947 |     b2bToast = document.querySelector('#b2b-toast'),
948 |     startButton = document.querySelector('#start-btn'),
949 |     pauseButton = document.querySelector('#pause-btn'),
950 |     restartButton = document.querySelector('#restart-btn'),
951 |     dasInput = document.querySelector('#das-input'),
952 |     arrInput = document.querySelector('#arr-input'),
953 |     sdfInput = document.querySelector('#sdf-input');
954 |     if (
955 |     !(
956 |         boardGrid &&
957 |         5 === nextGrids.length &&
958 |         holdGrid &&
959 |         scoreValue &&
960 |         highScoreValue &&
961 |         levelValue &&
962 |         linesValue &&
963 |         goalValue &&
964 |         comboToast &&
965 |         b2bToast &&
966 |         startButton &&
967 |         pauseButton &&
968 |         restartButton &&
969 |         dasInput &&
970 |         arrInput &&
971 |         sdfInput
972 |     )
973 |     )
974 |     throw new Error('Renderer targets are missing');
975 |     const boardCells = createCellGrid(boardGrid, 200, 'cell'),
976 |     nextCells = nextGrids.map((grid) => createCellGrid(grid, 16, 'preview-cell')),
977 |     holdCells = createCellGrid(holdGrid, 16, 'preview-cell');
978 |     ((dasInput.min = `${handlingRanges_dasMs.min}`),
979 |     (dasInput.max = `${handlingRanges_dasMs.max}`),
980 |     (arrInput.min = `${handlingRanges_arrMs.min}`),
981 |     (arrInput.max = `${handlingRanges_arrMs.max}`),
982 |     (sdfInput.min = `${handlingRanges_sdfG.min}`),
983 |     (sdfInput.max = `${handlingRanges_sdfG.max}`));

```

```

984 |     const paintPreview = (cells, pieceType) => {
985 |         if (
986 |             (cells.forEach((cell) => {
987 |                 cell.className = 'preview-cell';
988 |             })),
989 |             pieceType)
990 |         )
991 |             for (const point of PREVIEW_CELLS[pieceType]) {
992 |                 const cell = cells[4 * point.y + point.x];
993 |                 cell && (cell.className = `preview-cell ${toneClass(pieceType)}-is-active`);
994 |             }
995 |         };
996 |         let prevCombo = -1,
997 |             prevB2b = 0,
998 |             comboToastUntilMs = 0,
999 |             b2bToastUntilMs = 0,
1000 |             comboToastValue = 0,
1001 |             b2bToastValue = 0;
1002 |         return {
1003 |             render: (snapshot, highScore2) => {
1004 |                 const now = performance.now(),
1005 |                     locked = new Array(200).fill(null),
1006 |                     ghostMask = new Array(200).fill(!1),
1007 |                     activeMask = new Array(200).fill(!1);
1008 |                 for (let y = 0; y < snapshot.boardVisible.length; y += 1)
1009 |                     for (let x = 0; x < snapshot.boardVisible[y].length; x += 1)
1010 |                         locked[10 * y + x] = snapshot.boardVisible[y][x];
1011 |                 for (const point of snapshot.ghostCells) {
1012 |                     const index = 10 * point.y + point.x;
1013 |                     index ≥ 0 && index < ghostMask.length && (ghostMask[index] = !0);
1014 |                 }
1015 |                 for (const point of snapshot.activeCells) {
1016 |                     const index = 10 * point.y + point.x;
1017 |                     index ≥ 0 && index < activeMask.length && (activeMask[index] = !0);
1018 |                 }
1019 |                 for (let index = 0; index < boardCells.length; index += 1) {
1020 |                     const cell = boardCells[index];
1021 |                     cell.className = 'cell';
1022 |                     const lockedPiece = locked[index];
1023 |                     (lockedPiece && cell.classList.add(toneClass(lockedPiece), 'is-locked'),
1024 |                     lockedPiece || !ghostMask[index] || activeMask[index] || cell.classList.add('is-ghost'),
1025 |                     activeMask[index] &&
1026 |                     snapshot.activeType &&
1027 |                     cell.classList.add(toneClass(snapshot.activeType), 'is-active'));
1028 |                 }
1029 |                 for (let index = 0; index < nextCells.length; index += 1)
1030 |                     paintPreview(nextCells[index], snapshot.next[index] ?? null);
1031 |                 (paintPreview(holdCells, snapshot.hold),
1032 |                 setText(scoreValue, `${snapshot.score}`),
1033 |                 setText(highScoreValue, `${highScore2}`),
1034 |                 setText(levelValue, `${snapshot.level}`),
1035 |                 setText(linesValue, `${snapshot.lines}`),
1036 |                 setText(goalValue, `${snapshot.goal}`),
1037 |                 boardGrid.classList.toggle('is-paused', 'paused' === snapshot.status),
1038 |                 snapshot.combo > prevCombo &&
1039 |                 snapshot.combo > 0 &&
1040 |                 (((comboToastValue = snapshot.combo),
1041 |                 (comboToastUntilMs = now + 1200),
1042 |                 setText(comboToast, `COMBO x${comboToastValue}`)),
1043 |                 snapshot.b2bChain > prevB2b &&
1044 |                 snapshot.b2bChain > 0 &&
1045 |                 (((b2bToastValue = snapshot.b2bChain),
1046 |                 (b2bToastUntilMs = now + 1200),
1047 |                 setText(b2bToast, `B2B x${b2bToastValue}`)),
1048 |                 (comboToast.hidden = now ≥ comboToastUntilMs),
1049 |                 (b2bToast.hidden = now ≥ b2bToastUntilMs),
1050 |                 (prevCombo = snapshot.combo),
1051 |                 (prevB2b = snapshot.b2bChain));
1052 |             },
1053 |             bindControls: (bindings) => {
1054 |                 (startButton.addEventListener('click', bindings.onStart),

```

```

1055 |         pauseButton.addEventListener('click', bindings.onPause),
1056 |         restartButton.addEventListener('click', bindings.onRestart));
1057 |         const onHandlingInput = () => {
1058 |             bindings.onHandlingChange({
1059 |                 dasMs: Number(dasInput.value),
1060 |                 arrMs: Number(arrInput.value),
1061 |                 sdfG: Number(sdfInput.value),
1062 |             });
1063 |         };
1064 |         (dasInput.addEventListener('change', onHandlingInput),
1065 |         arrInput.addEventListener('change', onHandlingInput),
1066 |         sdfInput.addEventListener('change', onHandlingInput));
1067 |     },
1068 |     setHandlingInputs: (handling) => {
1069 |         ((dasInput.value = `${handling.dasMs}`),
1070 |         (arrInput.value = `${handling.arrMs}`),
1071 |         (sdfInput.value = `${handling.sdfG}`));
1072 |     },
1073 | };
1074 | })());
1075 | (renderer.setHandlingInputs(initialHandling),
1076 | renderer.bindControls({
1077 |     onStart: () => {
1078 |         engine.dispatch({ type: 'start' });
1079 |     },
1080 |     onPause: () => {
1081 |         engine.dispatch({ type: 'togglePause' });
1082 |     },
1083 |     onRestart: () => {
1084 |         engine.dispatch({ type: 'restart' });
1085 |     },
1086 |     onHandlingChange: (nextHandling) => {
1087 |         const normalized = normalizeHandling(nextHandling);
1088 |         (engine.dispatch({ type: 'setHandling', payload: normalized })),
1089 |         renderer.setHandlingInputs(normalized),
1090 |         ((handling) => {
1091 |             try {
1092 |                 const normalized = normalizeHandling(handling);
1093 |                 globalThis.localStorage?.setItem('tetris_handling_v1', JSON.stringify(normalized));
1094 |             } catch {}
1095 |         })(normalized));
1096 |     },
1097 | }));
1098 | const input = ({
1099 |     target: target,
1100 |     dispatch: dispatch,
1101 |     getHandling: getHandling,
1102 |     isRunning: isRunning,
1103 | }) => {
1104 |     const pressed = new Set();
1105 |     let horizontalOwner = null,
1106 |         dasElapsedMs = 0,
1107 |         arrElapsedMs = 0;
1108 |     const resetHorizontalTimers = () => {
1109 |         ((dasElapsedMs = 0), (arrElapsedMs = 0));
1110 |     },
1111 |     moveByOwner = () =>
1112 |         'left' === horizontalOwner
1113 |         ? dispatch({ type: 'moveLeft' })
1114 |         : 'right' === horizontalOwner && dispatch({ type: 'moveRight' }),
1115 |     handoffOwnerIfNeeded = () => {
1116 |         const leftPressed = pressed.has('ArrowLeft'),
1117 |             rightPressed = pressed.has('ArrowRight');
1118 |         if (!leftPressed || !rightPressed)
1119 |             return leftPressed
1120 |                 ? ((horizontalOwner = 'left'),
1121 |                 resetHorizontalTimers(),
1122 |                 void dispatch({ type: 'moveLeft' }))
1123 |                 : rightPressed
1124 |                 ? ((horizontalOwner = 'right'),
1125 |                 resetHorizontalTimers(),

```



```

1126 |         .void dispatch({ type: 'moveRight' }))
1127 |         .void (horizontalOwner = null);
1128 |     },
1129 |     onKeyDown = (event) => {
1130 |         if (
1131 |             HANDLED_CODES.has(event.code) &&
1132 |             (event.preventDefault(), !event.repeat && !pressed.has(event.code))
1133 |         )
1134 |             switch ((pressed.add(event.code), event.code)) {
1135 |                 case 'Enter':
1136 |                     return void dispatch({ type: 'start' });
1137 |                 case 'KeyP':
1138 |                     return void dispatch({ type: 'togglePause' });
1139 |                 case 'KeyR':
1140 |                     return void dispatch({ type: 'restart' });
1141 |                 case 'ArrowLeft':
1142 |                     return (
1143 |                         (horizontalOwner = 'left'),
1144 |                         resetHorizontalTimers(),
1145 |                         void dispatch({ type: 'moveLeft' })
1146 |                     );
1147 |                 case 'ArrowRight':
1148 |                     return (
1149 |                         (horizontalOwner = 'right'),
1150 |                         resetHorizontalTimers(),
1151 |                         void dispatch({ type: 'moveRight' })
1152 |                     );
1153 |                 case 'ArrowUp':
1154 |                 case 'KeyX':
1155 |                     return void dispatch({ type: 'rotateCW' });
1156 |                 case 'KeyZ':
1157 |                     return void dispatch({ type: 'rotateCCW' });
1158 |                 case 'KeyA':
1159 |                     return void dispatch({ type: 'rotate180' });
1160 |                 case 'Space':
1161 |                     return void dispatch({ type: 'hardDrop' });
1162 |                 case 'KeyC':
1163 |                 case 'ShiftLeft':
1164 |                 case 'ShiftRight':
1165 |                     return void dispatch({ type: 'hold' });
1166 |                 default:
1167 |                     return;
1168 |             }
1169 |     },
1170 |     onKeyUp = (event) => {
1171 |         HANDLED_CODES.has(event.code) &&
1172 |         (pressed.delete(event.code),
1173 |         'ArrowLeft' !== event.code || 'left' !== horizontalOwner
1174 |         ? 'ArrowRight' === event.code && 'right' === horizontalOwner && handoffOwnerIfNeeded()
1175 |         : handoffOwnerIfNeeded());
1176 |     };
1177 |     return (
1178 |         target.addEventListener('keydown', onKeyDown),
1179 |         target.addEventListener('keyup', onKeyUp),
1180 |         {
1181 |             update: (dtMs) => {
1182 |                 if (!isRunning() || !horizontalOwner) return;
1183 |                 const keyCode = 'left' === horizontalOwner ? 'ArrowLeft' : 'ArrowRight';
1184 |                 if (!pressed.has(keyCode)) return void handoffOwnerIfNeeded();
1185 |                 const handling = getHandling();
1186 |                 if (((dasElapsedMs += dtMs), !(dasElapsedMs < handling.dasMs)))
1187 |                     if (0 !== handling.arrMs)
1188 |                         for (
1189 |                             arrElapsedMs += dtMs;
1190 |                             arrElapsedMs >= handling.arrMs && ((arrElapsedMs -= handling.arrMs), moveByOwner());
1191 |                         );
1192 |                     else for (; moveByOwner(););
1193 |             },
1194 |             destroy: () => {
1195 |                 (target.removeEventListener('keydown', onKeyDown),
1196 |                 target.removeEventListener('keyup', onKeyUp));

```

```

1197 |     },
1198 |     isSoftDropActive: () => pressed.has('ArrowDown'),
1199 |   }
1200 | );
1201 | })(
1202 |   target: window,
1203 |   dispatch: (action) => engine.dispatch(action),
1204 |   getHandling: () => engine.getHandling(),
1205 |   isRunning: () => 'running' === engine.getSnapshot().status,
1206 | });
1207 | let lastTimestamp = performance.now();
1208 | const loop = (timestamp) => {
1209 |   const dtMs = Math.min(timestamp - lastTimestamp, 50);
1210 |   ((lastTimestamp = timestamp),
1211 |     input.update(dtMs),
1212 |     engine.setSoftDropActive(input.isSoftDropActive()),
1213 |     engine.tick(dtMs));
1214 |   const snapshot = engine.getSnapshot();
1215 |   (snapshot.score > highScore &&
1216 |     ((highScore = snapshot.score),
1217 |       ((score) => {
1218 |         try {
1219 |           const normalized = Math.max(0, Math.floor(score));
1220 |           globalThis.localStorage?.setItem('tetris_high_score_v1', `${normalized}`);
1221 |         } catch {}
1222 |       })(highScore)),
1223 |     renderer.render(snapshot, highScore),
1224 |     window.requestAnimationFrame(loop));
1225 | };
1226 | (renderer.render(engine.getSnapshot(), highScore),
1227 |   window.requestAnimationFrame(loop),
1228 |   window.addEventListener('beforeunload', () => {
1229 |     input.destroy();
1230 |   }));
1231 |

```

==== assets/index-DWoTo8jX.css (5855 B / 87cf11d43b33) ====

```

1 | :root{
2 |   --bg: oklch(100% 0 0deg);
3 |   --surface: oklch(98% 0 0deg);
4 |   --line: oklch(30% 0 0deg);
5 |   --line-soft: oklch(85% 0 0deg);
6 |   --text: oklch(20% 0 0deg);
7 |   --muted: oklch(45% 0 0deg);
8 |   --ghost: oklch(75% 0 0deg / 75%);
9 |   --tone-i: oklch(82% 0.15 220deg);
10 |  --tone-o: oklch(90% 0.15 98deg);
11 |  --tone-t: oklch(70% 0.15 315deg);
12 |  --tone-s: oklch(82% 0.15 150deg);
13 |  --tone-z: oklch(72% 0.15 25deg);
14 |  --tone-j: oklch(66% 0.15 255deg);
15 |  --tone-l: oklch(78% 0.15 70deg);
16 |  --radius: 0.5rem;
17 |  --gap: 0.75rem;
18 |  --font: 'Segoe UI', 'Noto Sans KR', 'Apple SD Gothic Neo', 'Malgun Gothic', sans-serif;
19 | }
20 | *,
21 | *:before,
22 | *:after{
23 |   box-sizing: border-box;
24 | }
25 | *{
26 |   margin: 0;
27 | }
28 | html,
29 | body{
30 |   min-height: 100%;
31 | }
32 | body{
33 |   line-height: 1.5;
34 |   -webkit-font-smoothing: antialiased;

```

```

35 | }
36 | img,
37 | picture,
38 | svg,
39 | video {
40 |     display: block;
41 |     max-width: 100%;
42 | }
43 | input,
44 | button,
45 | textarea,
46 | select {
47 |     font: inherit;
48 | }
49 | ul,
50 | ol {
51 |     list-style: none;
52 |     padding: 0;
53 | }
54 | body {
55 |     background: var(--bg);
56 |     color: var(--text);
57 |     font-family: var(--font);
58 | }
59 | #app {
60 |     min-height: 100vh;
61 |     padding: 1rem;
62 |     display: grid;
63 |     place-items: center;
64 | }
65 | .layout {
66 |     width: min(72rem, 100%);
67 |     min-height: min(96vh, 60rem);
68 |     display: grid;
69 |     grid-template-columns: 18rem minmax(0, 26rem) 18rem;
70 |     column-gap: 0.45rem;
71 |     row-gap: var(--gap);
72 |     justify-content: center;
73 |     align-items: start;
74 | }
75 | .panel-card {
76 |     border: 1px solid var(--line);
77 |     border-radius: var(--radius);
78 |     background: var(--surface);
79 | }
80 | .board-anchor {
81 |     position: relative;
82 |     width: 100%;
83 |     max-width: 26rem;
84 |     margin: auto;
85 |     grid-column: 2;
86 |     grid-row: 1;
87 | }
88 | .tetris-board {
89 |     width: 100%;
90 |     aspect-ratio: 10/20;
91 |     margin: 0;
92 |     position: relative;
93 |     border: 1px solid var(--line);
94 |     border-radius: 0.3rem;
95 |     padding: 0.2rem;
96 |     display: grid;
97 |     grid-template-columns: repeat(10, minmax(0, 1fr));
98 |     grid-template-rows: repeat(20, minmax(0, 1fr));
99 |     gap: 0;
100 |     line-height: 0;
101 |     overflow: hidden;
102 | }
103 | .tetris-board.is-paused:after {
104 |     content: 'PAUSED';
105 |     position: absolute;

```

```

106 | inset: 0;
107 | display: grid;
108 | place-items: center;
109 | font-size: 1.5rem;
110 | font-weight: 800;
111 | letter-spacing: 0.08em;
112 | color: var(--text);
113 | text-shadow: 0 0 0.4rem oklch(100% 0 0deg / 0.7);
114 | pointer-events: none;
115 | z-index: 2;
116 | }
117 | .cell,
118 | .preview-cell {
119 |   margin: 0;
120 |   border: 0;
121 |   padding: 0;
122 |   display: block;
123 |   width: 100%;
124 |   height: 100%;
125 |   background: var(--surface);
126 | }
127 | .cell.is-ghost {
128 |   background: var(--ghost);
129 | }
130 | .tone-i {
131 |   background: var(--tone-i);
132 | }
133 | .tone-o {
134 |   background: var(--tone-o);
135 | }
136 | .tone-t {
137 |   background: var(--tone-t);
138 | }
139 | .tone-s {
140 |   background: var(--tone-s);
141 | }
142 | .tone-z {
143 |   background: var(--tone-z);
144 | }
145 | .tone-j {
146 |   background: var(--tone-j);
147 | }
148 | .tone-l {
149 |   background: var(--tone-l);
150 | }
151 | .button-row {
152 |   display: flex;
153 |   gap: 0.35rem;
154 |   flex-wrap: wrap;
155 |   grid-column: 2;
156 |   grid-row: 2;
157 |   justify-content: center;
158 | }
159 | button,
160 | input {
161 |   border: 1px solid var(--line);
162 |   border-radius: 0.35rem;
163 |   padding: 0.35rem 0.5rem;
164 |   background: var(--surface);
165 |   color: var(--text);
166 | }
167 | button {
168 |   cursor: pointer;
169 | }
170 | .side-panel {
171 |   display: grid;
172 |   grid-template-rows: repeat(2, minmax(0, auto));
173 |   gap: var(--gap);
174 | }
175 | .side-panel-left {
176 |   justify-items: stretch;

```

```

177 |   grid-column: 1;
178 |   grid-row: 1 / span 2;
179 | }
180 | .side-panel-right {
181 |   justify-items: stretch;
182 |   grid-column: 3;
183 |   grid-row: 1 / span 2;
184 |   border: 1px solid var(--line);
185 |   border-radius: var(--radius);
186 |   background: var(--surface);
187 |   padding: 0.55rem;
188 | }
189 | .panel-card {
190 |   padding: 0.55rem;
191 |   display: grid;
192 |   gap: 0.35rem;
193 |   align-content: start;
194 | }
195 | .panel-card h2 {
196 |   font-size: 0.9rem;
197 | }
198 | .stats-hud {
199 |   display: grid;
200 |   gap: 0.2rem;
201 |   align-content: start;
202 | }
203 | .stats-hud-title {
204 |   font-size: 0.82rem;
205 |   color: var(--muted);
206 | }
207 | .stats-hud-score {
208 |   font-size: 2rem;
209 |   font-weight: 800;
210 |   line-height: 1;
211 | }
212 | .stats-hud-high-score {
213 |   font-size: 0.84rem;
214 |   color: var(--muted);
215 |   display: flex;
216 |   justify-content: space-between;
217 |   gap: 0.4rem;
218 | }
219 | .stats-divider {
220 |   border-top: 1px solid var(--line);
221 |   margin: 0.24rem 0 0.18rem;
222 | }
223 | .stats-mini-list {
224 |   display: grid;
225 |   gap: 0.2rem;
226 | }
227 | .stats-mini-item {
228 |   font-size: 0.82rem;
229 |   color: var(--muted);
230 |   display: flex;
231 |   justify-content: space-between;
232 |   gap: 0.4rem;
233 | }
234 | .preview-panel {
235 |   display: grid;
236 |   gap: 0.35rem;
237 |   align-content: start;
238 | }
239 | .preview-panel h2 {
240 |   font-size: 0.9rem;
241 | }
242 | .settings-panel label {
243 |   font-size: 0.82rem;
244 |   color: var(--muted);
245 | }
246 | #event-toast {
247 |   position: absolute;

```

```

248 | left: 0;
249 | bottom: 0;
250 | transform: translate(calc(-100% - 0.35rem));
251 | z-index: 30;
252 | display: grid;
253 | gap: 0.28rem;
254 | pointer-events: none;
255 | }
256 | .event-toast-item {
257 |   margin: 0;
258 |   padding: 0.2rem 0.45rem;
259 |   border: 1px solid var(--line);
260 |   border-radius: 0.3rem;
261 |   background: var(--surface);
262 |   color: var(--text);
263 |   font-size: 1rem;
264 |   font-weight: 800;
265 |   letter-spacing: 0.04em;
266 | }
267 | .preview-grid {
268 |   width: 7rem;
269 |   aspect-ratio: 1/1;
270 |   border: 1px solid var(--line);
271 |   border-radius: 0.3rem;
272 |   padding: 0.2rem;
273 |   display: grid;
274 |   grid-template-columns: repeat(4, minmax(0, 1fr));
275 |   grid-template-rows: repeat(4, minmax(0, 1fr));
276 |   gap: 0;
277 |   line-height: 0;
278 |   overflow: hidden;
279 | }
280 | .next-list {
281 |   display: grid;
282 |   gap: 0.28rem;
283 |   justify-items: start;
284 | }
285 | .next-grid {
286 |   width: 4.8rem;
287 | }
288 | .next-grid,
289 | #hold-grid {
290 |   border: 0;
291 |   background: #bebebe;
292 | }
293 | .next-grid .preview-cell:not(.is-active),
294 | #hold-grid .preview-cell:not(.is-active) {
295 |   background: #bebebe;
296 | }
297 | @media (max-width: 980px) {
298 |   .layout {
299 |     min-height: auto;
300 |     grid-template-columns: 1fr;
301 |   }
302 |   .tetris-board,
303 |   .board-anchor,
304 |   .button-row,
305 |   .side-panel-left,
306 |   .side-panel-right {
307 |     grid-column: auto;
308 |     grid-row: auto;
309 |   }
310 |   .side-panel {
311 |     grid-template-columns: 1fr;
312 |     grid-template-rows: none;
313 |   }
314 | }
315 | @media (max-width: 640px) {
316 |   .side-panel {
317 |     grid-template-columns: 1fr;
318 |   }

```

```

319 |     .stats-hud-score-{
320 |     |     font-size: 1.7rem;
321 |     }
322 |     .event-toast-item-{
323 |     |     font-size: 0.9rem;
324 |     }
325 | }
326 |

```

==== index.html (3255 B / 553f0ec6681f) ====

```

1 | <!doctype html>
2 | <html lang="ko">
3 |   <head>
4 |     <meta charset="UTF-8" />
5 |     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6 |     <title>Tetris</title>
7 |     <script type="module" crossorigin src="/assets/index-CNeejhjd.js"></script>
8 |     <link rel="stylesheet" crossorigin href="/assets/index-DWoTo8jX.css" />
9 |   </head>
10 |   <body>
11 |     <div id="app">
12 |       <main class="layout">
13 |         <aside class="side-panel side-panel-left" aria-label="왼쪽 정보 패널">
14 |           <section class="stats-hud" aria-label="점수 패널">
15 |             <p class="stats-hud-title">Score</p>
16 |             <p class="stats-hud-score"><span id="score-value">0</span></p>
17 |             <p class="stats-hud-high-score">High Score <span id="high-score-value">0</span></p>
18 |             <div class="stats-divider" aria-hidden="true"></div>
19 |             <div class="stats-mini-list">
20 |               <p class="stats-mini-item">Level <span id="level-value">1</span></p>
21 |               <p class="stats-mini-item">Lines <span id="lines-value">0</span></p>
22 |               <p class="stats-mini-item">Goal <span id="goal-value">10</span></p>
23 |             </div>
24 |           </section>
25 |
26 |           <section class="panel-card settings-panel" aria-label="핸들링 설정">
27 |             <h2>Handling</h2>
28 |             <label for="das-input">DAS (ms)</label>
29 |             <input id="das-input" type="number" step="1" />
30 |
31 |             <label for="arr-input">ARR (ms)</label>
32 |             <input id="arr-input" type="number" step="1" />
33 |
34 |             <label for="sdf-input">SDF (G)</label>
35 |             <input id="sdf-input" type="number" step="0.5" />
36 |           </section>
37 |         </aside>
38 |
39 |         <div class="board-anchor">
40 |           <div
41 |             id="board-grid"
42 |             class="tetris-board"
43 |             role="img"
44 |             aria-label="10열 20행 게임 그리드"
45 |           ></div>
46 |           <div id="event-toast" aria-live="polite" aria-atomic="true">
47 |             <p id="combo-toast" class="event-toast-item" hidden></p>
48 |             <p id="b2b-toast" class="event-toast-item" hidden></p>
49 |           </div>
50 |         </div>
51 |         <div class="button-row">
52 |           <button id="start-btn" type="button">Start</button>
53 |           <button id="pause-btn" type="button">Pause</button>
54 |           <button id="restart-btn" type="button">Restart</button>
55 |         </div>
56 |
57 |         <aside class="side-panel side-panel-right" aria-label="오른쪽 미리보기 패널">
58 |           <section class="preview-panel" aria-label="다음 블록 패널">
59 |             <h2>Next</h2>
60 |             <div id="next-list" class="next-list">
61 |               <div class="preview-grid next-grid" data-next-index="0"></div>

```

```

62 | . . . . . <div class="preview-grid next-grid" data-next-index="1"></div>
63 | . . . . . <div class="preview-grid next-grid" data-next-index="2"></div>
64 | . . . . . <div class="preview-grid next-grid" data-next-index="3"></div>
65 | . . . . . <div class="preview-grid next-grid" data-next-index="4"></div>
66 | . . . . . </div>
67 | . . . . . </section>
68 |
69 | . . . . . <section class="preview-panel" aria-label="홀드 블록 패널">
70 | . . . . . <h2>Hold</h2>
71 | . . . . . <div id="hold-grid" class="preview-grid"></div>
72 | . . . . . </section>
73 | . . . . . </aside>
74 | . . . . . </main>
75 | . . . . . </div>
76 | . . . . . </body>
77 | </html>
78 |

```